

Code Review Nyox:

Data da Revisão: 18 de Novembro de 2025

Introdução

Este documento apresenta uma revisão de código detalhada de diversos componentes do projeto de Catálogo de Imóveis, abrangendo o *backend* (rotas API), o *frontend* (componentes React) e a camada de testes unitários.

O objetivo da análise é identificar pontos fortes na arquitetura, garantir o uso de boas práticas de desenvolvimento e, principalmente, validar a integridade e a segurança do sistema de autenticação e manipulação de dados sensíveis.

Sumário das Análises

Camada	Arquivo Principal	Foco da Análise	Status Geral
Backend (API)	<code>imoveis.js</code> (Rotas)	CRUD, Geocodificação (LocationIQ), Comunicação com Supabase, Segurança de Rota.	Vulnerabilidade de Segurança Pendente
Frontend (Componentes)	<code>PainelAdm.jsx</code> , <code>Create.jsx</code> , etc.	Autenticação (JWT), Validação de Formulários, UX, Otimização de Imagens/Mapas.	Forte, com Ajustes de UX/Segurança
Testes (Frontend)	<code>Create.test.js</code> , <code>Login.test.js</code>	Cobertura de fluxo, Mocks (CEP, Fetch), Simulação de Erros.	Alta Cobertura e Qualidade

I. Backend - Rotas e Lógica de Negócio

O arquivo `imoveis.js` define as rotas para o gerenciamento de imóveis, incluindo integração com serviços externos (LocationIQ) e o banco de dados (Supabase).

1.1. Código da Rota de Imóveis (`imoveis.js`)

JavaScript

```

import dotenv from 'dotenv';
import path from 'path';
dotenv.config({ path: path.resolve(process.cwd(), '.env') });
import { Router } from "express";
import multer from "multer";
import supabase from '../Connection/Supabase.js';
import uploadFile from "../Upload/imageUpload.js";
const upload = multer({ storage: multer.memoryStorage() });

const router = Router();

// ... [Swagger Documentation]

router.post('/register', upload.array("imagens", 5), async (req, res) => {
  try {

    const body = {
      descricao: req.body.descricao,
      endereco: req.body.endereco,
      bairro: req.body.bairro,
      cidade: req.body.cidade,
      estado: req.body.estado,
      cep: Number(req.body.cep),
      tipo: req.body.tipo,
      quartos: Number(req.body.quartos),
      banheiros: Number(req.body.banheiros),
      vagas_garagem: Number(req.body.vagas_garagem),
      area_total: Number(req.body.area_total),
      valor: Number(req.body.valor),
    };

    let imageUrls = []
    // ... [Image upload logic]
    body.imagens = imageUrls.filter(Boolean)

    const { data, error } = await supabase
      .from("Imoveis")
      .insert([body]);
    if (error) {
      console.error("Erro do Supabase:", error);
      return res.status(400).json({ success: false, message: 'Erro ao inserir imóvel' });
    } else {
      return res.status(200).json({ success: true, message: 'Imóvel inserido com sucesso',
data: data });
    }
  } catch (err) {
    console.error(err);
  }
}

```

```

    return res.status(500).json({ success: false, message: 'Erro interno do servidor' });
  }
});

// ... [Other CRUD and list routes]

router.get('/coords/:q', async(req, res) =>{
  // ... [LocationIQ geocoding logic]
});

export default router;

```

1.2. Sugestões de Melhoria (Backend)

Área	Sugestão
Segurança (CRÍTICA)	Implementar Middleware de Autenticação/Autorização (ADM) em todas as rotas de modificação (<code>/register</code> , <code>/update</code> , <code>/delete</code> , etc.). Atualmente, essas rotas estão desprotegidas e acessíveis por qualquer cliente.
Robustez	Adicionar validação de <i>input</i> nos campos numéricos no backend antes da inserção no banco de dados.
Performance	O uso de <code>Promise.all</code> para uploads de imagem é uma excelente prática que maximiza a performance.

II. Frontend - Componentes React

Os componentes de frontend implementam a interface do usuário, a lógica de autenticação (guarda de rota) e o consumo das APIs.

2.1. Componente `PainelAdm.jsx` (Guarda de Rota e Autorização)

```

JavaScript
import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { FontAwesomeIcon, faTrash, faEdit } from '@fortawesome/free-solid-svg-icons'
// ...
import jwt_decode from "jwt-decode";

function PainelAdm() {

```

```
// ...
useEffect(() => {
  const token = localStorage.getItem("token");
  if (!token) { /* ... redirect to /login */ }

  try {
    const decoded = jwt_decode(token);
    const now = Math.floor(Date.now() / 1000);
    if (decoded.exp && decoded.exp < now) { /* ... remove token and redirect */ }

    // Excelente: Checagem de Autorização no Frontend
    if (decoded.type !== "adm") {
      Alert("Seu usuário não tem permissões necessárias!", 'Erro!', 'error')
      navigate("/login");
      return;
    }
    // ...
  } catch (err) { /* ... handle token error */ }
}, []);

// ... (listalmoveis and handleDelete functions)
}
export default PaineAdm;
```

Sugestões de Melhoria (**PaineAdm.jsx**)

- As funções de modificação de dados (**handleDelete**) e listagem de imóveis **devem ser atualizadas** para enviar o **cabeçalho Authorization: Bearer <token>** nas requisições, cumprindo a regra de segurança do backend.
- A implementação da lógica de **Guarda de Rota** e verificação do tipo (**type: 'adm'**) é uma **prática de segurança exemplar** no lado do cliente.

2.2. Componente **Create.jsx** (Cadastro de Imóveis)

JavaScript

```
function Create() {
  // ... [State and useEffect for Guard/Cleanup]
  // ... [consultarCEP and handleImageChange logic]

  const handleSubmit = async (e) => {
    e.preventDefault();
    // ... [Form data and FormData creation]

    try {
      const res = await fetch('http://localhost:8800/imoveis/register', {
        method: 'POST',
        body: payload,
        // FALTA AQUI: headers: { 'Authorization': 'Bearer <token>' }
      });
    }
  }
}
```

```

    // ...
  } catch (error) {
    await Alert('Erro ao conectar ao servidor','Erro!','error');
  }
};
// ...
}
export default Create;

```

Sugestões de Melhoria (**Create.jsx**)

- **Ajuste de Segurança:** É mandatório adicionar o cabeçalho **Authorization** na requisição **POST** de **handleSubmit** para a rota **/imoveis/register**.
- A lógica de **criação de previews** e a **limpeza de memória** (**URL.revokeObjectURL**) são destaques de **alta qualidade** e performance.

2.3. Outros Componentes (UX e Dados Públicos)

Arquivo	Destaque Positivo	Sugestão de Melhoria
AnuncioPage.jsx	Uso de Intl.NumberFormat para formatação de moeda (R\$).	A seção de "Sugestões" exibe o mesmo imóvel repetidamente; deve ser atualizada para carregar dados dinâmicos.
MapView.jsx	Implementação de um mecanismo de cache local para coordenadas (48h), reduzindo a dependência de serviços externos.	O uso do subcomponente MoveMap é uma solução elegante para o problema de centralização de mapas no React/Leaflet.
AnuncioCard.jsx	Design limpo, bom uso de ícones (react-icons/fa).	Implementar a formatação de valor via Intl.NumberFormat para exibir o preço no padrão R\$ XXX.XXX,XX.
Alert.jsx	Padronização de mensagens de alerta usando sweetalert2 , melhorando a UX.	Remover importações não utilizadas (React , icon , withReactContent).

III. Testes Frontend (Cobertura e Qualidade)

Os testes usam a biblioteca **Vitest** e o **Testing Library** para garantir a funcionalidade da aplicação.

3.1. Código dos Testes (**Create.test.js** e **Login.test.js**)

JavaScript

// Exemplo de Teste - Create.test.js

```
describe("Create Page", () => {  
  // ...  
  it("deve enviar formulário com sucesso", async () => {  
    // ...  
    await waitFor(() => {  
      expect(global.fetch).toHaveBeenCalledWith(  
        "http://localhost:8800/moveis/register",  
        expect.objectContaining({ method: "POST", body: expect.any(FormData) })  
      );  
    });  
  });  
});
```

// Exemplo de Teste - Login.test.js

```
describe("Login Page", () => {  
  it("deve realizar login com sucesso", async () => {  
    // ...  
    await waitFor(() => {  
      expect(localStorage.getItem("token")).toBe("abc123");  
    });  
  });  
});
```

3.2. Sugestões de Melhoria (Testes)

- **Qualidade:** A utilização de funções auxiliares (**fill**, **submit**, **fillAndSubmit**) torna os testes extremamente **legíveis e de fácil manutenção**.
- **Cobertura Pendente:** Adicionar testes que confirmem a inclusão do cabeçalho **Authorization: Bearer <token>** nas requisições de submissão de formulário (**Create.test.js**).
- Incluir testes de validação de *input* para campos numéricos/vazios no frontend.

IV. Conclusão e Próximos Passos (Foco na Segurança)

O projeto demonstra uma **arquitetura moderna** (React, Express, JWT) e um forte compromisso com a qualidade através de testes e técnicas avançadas (caching de mapa, limpeza de memória).

Ação Mais Importante

A principal prioridade para garantir a integridade do sistema é a segurança do servidor:

A implementação do *Middleware* de Autenticação/Autorização no Backend é essencial para proteger rotas críticas de modificação de dados (`/imoveis/register`, `/delete`, etc.) de acessos não autorizados, complementando a excelente **Guarda de Rota** já implementada no frontend.